Shri Vile Parle Kelavani Mandal's
DWARKADAS J.SANGHVI COLLEGE OF ENGINEERING
(Empowered Autonomous College Affiliated to the University of Mumbai)

Department of Artificial Intelligence (AI) and Data Science

Learnova: Redesigning Outdated Lecture Presentations into Modern, Illustrated and Interactive Teaching Decks

Interdisciplinary Project Report

Swayam Devendra Panchal (60018240013)

Kawaljeet Singh Bharaj (60018240016)

Jeevesh Bharat Bodhani (60018240096)

Rishit Pankajkumar Lahoti (6001840101)

Under the guidance of the Department of Artificial Intelligence and Data Science (Head of Department: Dr. Pratik Kanani), Dwarkadas J. Sanghvi College of Engineering, Mumbai — October 2026

Contents

Certificate 2

Acknowledgement 3

Abstract 3

1 Introduction 5
- 1.1 Background . . . 5
- 1.2 Motivation . . . 5
- 1.3 Problem Statement . . . 6
- 1.4 Objectives . . . 6
- 1.5 Scope . . . 6
- 1.6 Organisation of the Report . . . 7

2 Literature Survey 7
- 2.1 Automatic Generation of Slides from Documents . . . 7
- 2.2 Faithfulness of AI-Generated Text . . . 8
- 2.3 Slide Design and Learning . . . 8
- 2.4 Worked Examples and Interactive Visualisation . . . 8
- 2.5 Automatic Visual Design and Layout . . . 9
- 2.6 Commercial Presentation Tools . . . 9
- 2.7 Comparison and Research Gap . . . 9

3 Requirements Analysis and Feasibility 10
- 3.1 Stakeholders . . . 10
- 3.2 Functional Requirements . . . 10
- 3.3 Non-Functional Requirements . . . 11
- 3.4 Hardware and Software Requirements . . . 12
- 3.5 Feasibility Study . . . 12

4 System Design 12
- 4.1 Architecture Overview . . . 12
- 4.2 Data Flow . . . 13
- 4.3 Module Design . . . 14
- 4.4 Database Design . . . 15
- 4.5 API Design . . . 16
- 4.6 User Interface Design . . . 16

5 Methodology and Algorithms 16
- 5.1 Recovering the Structure of a Slide Deck . . . 17
- 5.2 Content Decision Engine . . . 18
- 5.3 Pedagogical Slide Fitness (PSF) . . . 19

5.4 CLASS: Load-Optimal Slide Segmentation . . . 20
5.5 Layout Without Broken Words . . . 20
5.6 Illustrations with Two Fallbacks . . . 21
5.7 Step-by-Step Explainers . . . 22
5.8 Interactive Laboratories . . . 22
5.9 Grounded Language-Model Assistance . . . 22
5.10 Checkpoint Quizzes . . . 23

6 Implementation **23**
6.1 Technology Stack . . . 23
6.2 Project Structure . . . 23
6.3 Key Modules . . . 24
6.4 User Interface . . . 24
6.5 Testing . . . 26
6.6 Improvements after testing on the review decks . . . 26

7 Results and Evaluation **28**
7.1 Evaluation Setup . . . 28
7.2 Visual Decision Accuracy . . . 29
7.3 Fidelity, Layout and Speed . . . 29
7.4 Before and After . . . 31
7.5 PSF and CLASS . . . 33
7.6 Step-by-Step Explainers . . . 34
7.7 Illustrations . . . 34

8 How Learnova Solves the Problem Differently **35**

9 Conclusion and Future Scope **35**
9.1 Conclusion . . . 35
9.2 Limitations . . . 36
9.3 Future Scope . . . 36

References **36**

Appendix A: Sample Outputs **38**

Appendix B: Project Timeline **38**

Certificate

This is to certify that the project report entitled **"Learnova: Redesigning Outdated Lecture Presentations into Modern, Illustrated and Interactive Teaching Decks"** is a bonafide record of the Interdisciplinary Project work carried out by Swayam Devendra Panchal (60018240013), Kawaljeet

Singh Bharaj (60018240016), Jeevesh Bharat Bodhani (60018240096) and Rishit Pankajkumar Lahoti (6001840101) of the Department of Artificial Intelligence and Data Science, Dwarkadas J. Sanghvi College of Engineering, Mumbai, during the academic year 2025–2026 and the odd semester of 2026–2027, under our guidance and supervision.

Project Mentor: _____	Head of Department: Dr. Pratik Kanani
Signature: _____	Signature: _____
Date: _____	Date: _____

Acknowledgement

We express our sincere gratitude to Dr. Pratik Kanani, Head of the Department of Artificial Intelligence and Data Science, for providing the facilities, encouragement and academic direction that made this project possible. We are grateful to our project mentor for regular reviews, for pointing us towards real lecture material when our early prototype worked only on clean sample files, and for the critical questions that shaped the design of the system. We thank the faculty members of the department whose lecture decks we used to test Learnova, and our classmates who tried early versions of the application and told us honestly what confused them. Finally, we thank our families for their patience during the many evenings and weekends that went into this work.

Abstract

Lecture slides are the main study material in most engineering courses, yet a large share of the decks in use have been reused for years. They are dense bullet lists; equations, graphs and diagrams are pasted as pictures; many slides have no title; and nothing on the slide lets a student check their understanding. Redesigning a deck by hand takes a teacher several hours per lecture, and the artificial-intelligence presentation tools now available solve a different problem: they generate new slides from a prompt, and in doing so they paraphrase the teacher's points, merge or drop list items and lose the original figures. There is no tool that takes the deck a teacher already uses and improves it without changing what it says.

This report presents **Learnova**, a web platform that rebuilds an existing lecture deck — a PowerPoint file, a slide PDF, Word notes or a typed topic — into an editable PowerPoint presentation, an interactive web deck and a study pack, while keeping every point of the original. The system reads slide decks page by page, recovering titles, bullet levels, tables and figures, and uses optical character recognition only when a picture-only slide can be read reliably. An explainable rule-based Content Decision Engine chooses a suitable visual form for every slide — a process flow, a timeline, a comparison table,

a definition, a chart or cards — and records the reason for its choice. A layout engine measures text glyph by glyph so that no word is ever broken across lines. Every topic slide receives an illustration: the teacher's own figure, a diagram written by a large language model and validated before use, or an offline illustration; each illustration is published as an HTML page linked from the PowerPoint. Checkpoint quizzes, exact step-by-step solutions to problems and interactive laboratories for difficult topics turn reading into practice. Two original methods support the design: Pedagogical Slide Fitness (PSF), a slide-quality metric derived from cognitive-load and multimedia-learning theory, and CLASS, a dynamic program that splits an overloaded slide into consecutive slides of minimum total cognitive load.

Learnova was evaluated on gold sets of 101 labelled slides and on seven real course decks of 15 to 216 slides. The decision engine selected an appropriate visual for 96.8–97.4% of slides in under one millisecond per slide. The rebuilt decks retained 98–100% of the numeric and named facts of their sources, the number of untitled slides across five decks fell from 83 to zero, and no slide was flagged for overflowing or broken text. The step-by-step solvers answered all 26 problems of a held-out benchmark correctly, and CLASS was never worse than even splitting while being strictly better on 37% of overloaded sections. The system works with free API keys when they are available and completely offline when they are not.

Keywords: educational technology, presentation redesign, document structure recovery, explainable artificial intelligence, cognitive load, large language models, interactive learning.

1. Introduction

1.1 Background

Slides have become the backbone of classroom teaching in engineering colleges. A typical course is taught from a set of decks prepared once and refined slowly over several years; the same decks are shared with students and become their primary revision material before internal and university examinations. When we collected the decks used in three of our own courses — Natural Language Processing, Machine Learning, and Image Processing and Computer Vision — we found the same pattern in almost all of them. Slides carried long bullet lists that read like notes rather than explanations. Important equations, derivations and diagrams had been copied from textbooks as screenshots, so they could not be searched, resized or explained. Many slides had no title, or a title that only repeated the topic name, so a student flipping through the deck could not tell what each slide was supposed to teach. None of the decks contained anything a student could interact with: there were no questions to check understanding, no worked steps that a beginner could follow, and no way to try an algorithm on a different input.

These observations match what educational research has reported for a long time. Cognitive load theory explains that working memory can hold only a few new elements at once, so a slide that presents twelve unconnected bullet points overloads the learner. Mayer's principles of multimedia learning show that people learn better when a picture appears next to the words that explain it, when irrelevant material is removed and when the structure of the material is signalled clearly. Studies of the assertion–evidence approach show that slides with a full-sentence headline supported by a visual are understood and remembered better than slides with a topic title and bullets. The problem, therefore, is not that teachers do not know what a good slide looks like; it is that redesigning hundreds of existing slides by hand is not practical.

1.2 Motivation

Two developments made this project timely. First, large language models can now write fluent text, propose titles and even draw simple diagrams, which suggested that much of the redesign work could be automated. Second, the tools that apply these models to presentations — Gamma, Microsoft Copilot in PowerPoint, Gemini in Google Slides and Canva Magic Design — had become widely available. When we tried them on our lecture decks, however, the results were disappointing for teaching. The tools produced attractive slides, but they rewrote the teacher's wording, merged or dropped items from lists, lost the original figures and equations, and offered no way to check what had changed. They also required a paid subscription and a continuous internet connection. For a study aid, a deck that looks better but silently loses the point that will be asked in the examination is worse than the original.

Our motivation was to build a tool that a teacher could trust: one that improves how a deck is presented while guaranteeing that what the deck says is preserved, that explains each of its design decisions, and

that adds the interactive elements research recommends — all without depending on paid services.

1.3 Problem Statement

Faculty members need a way to turn their existing lecture decks into modern, readable and interactive teaching material without redesigning them by hand and without losing or altering their content. Existing AI presentation tools generate new decks from prompts, are not faithful to the source, ignore picture-only and equation slides, provide no interaction and depend on paid cloud services. The problem addressed by this project is therefore:

> *To design and build a system that takes an existing lecture deck in any common format and rebuilds it as an illustrated, interactive and editable deck that preserves every point of the source, explains its design decisions, and works both with and without AI services.*

1.4 Objectives

The project set out to achieve the following objectives:

1. **Faithful reading of real lecture material.** Read PowerPoint files, slide PDFs, Word documents and typed topics, and recover the title, bullet levels, tables, figures and picture-only slides of every page, including decks with symbol-font bullets, repeated logos and missing titles.
2. **Explainable visual redesign.** Choose a suitable visual form for every slide using inspectable rules, and record the reason and confidence of every choice so that a teacher can review it.
3. **Content preservation.** Keep all of the source content: no item dropped, no fact invented and no word broken across lines, and measure this on real decks.
4. **Illustration on every slide.** Give every topic slide a relevant picture, preferring the teacher's own figures, and make each illustration available as a page that can be opened from the PowerPoint.
5. **Interactivity.** Add checkpoint quizzes, exact step-by-step solutions for problems and interactive laboratories for difficult topics.
6. **Dependability.** Use AI services where they improve the result and can be checked, and fall back to offline logic whenever a key is missing, rejected or exhausted.

1.5 Scope

Learnova handles lecture material written in English for engineering and science subjects. The inputs supported are PowerPoint (.pptx) files, PDF files exported from slide software, Word (.docx) documents, Markdown or plain text, and short typed topics. The outputs are an editable PowerPoint file using native shapes, a self-contained web deck that runs in any modern browser, a study pack (flashcards in CSV and Anki formats, a quiz bank, a cheat sheet and narration notes) and a view-only share link. The system is a decision-support tool for teachers: it does not replace their judgement, every

slide it produces can be edited, and every automatic decision is shown with its reason. Handwritten notes, scanned textbooks and languages other than English are outside the present scope.

1.6 Organisation of the Report

Chapter 2 reviews research on automatic slide generation, faithfulness of AI-generated text, slide design and learning, and the commercial tools available today, and identifies the gap Learnova addresses. Chapter 3 analyses the requirements and the feasibility of the system. Chapter 4 describes the architecture and the design of each subsystem. Chapter 5 presents the methods and algorithms we developed, including the Pedagogical Slide Fitness metric and the CLASS segmentation algorithm. Chapter 6 describes the implementation, and Chapter 7 reports the evaluation on gold sets and real decks. Chapter 8 discusses how Learnova differs from existing approaches, and Chapter 9 concludes the report with limitations and future work.

2. Literature Survey

2.1 Automatic Generation of Slides from Documents

Research on generating presentations automatically has mostly treated the task as summarisation of a scientific paper. **PPSGen** (Hu and Wan, IEEE Transactions on Knowledge and Data Engineering, 2015) learned a regression model that scores how important each sentence of a paper is and then used integer linear programming to select sentences and arrange them into slides. The approach produced reasonable slide text for papers, but it is extractive summarisation: content judged less important is left out, and it produces no visuals beyond the bullet text.

DOC2PPT (Fu, Wang, McDuff and Song, AAAI, 2022) introduced a hierarchical neural model that summarises each section of a scientific document, selects figures from the paper and predicts a slide layout. It was trained on thousands of paper–slide pairs. DOC2PPT showed that figures and layout can be handled jointly with text, but it requires large training data, its abstractive summaries can change or omit facts, and its input must be a scientific paper rather than an existing deck.

D2S (Sun, Hou, Wang, Zhang and Wang, NAACL, 2021) took a different view: given slide titles, it treated each title as a query, retrieved relevant passages from the paper and summarised them into the slide body. This made the generated slides more focused, but D2S works only with text, produces no figures, quizzes or interaction, and again assumes a paper as input.

Across these systems, the input is a long document and the output is a shorter one. None of them starts from an existing deck, and all of them trade completeness for brevity.

2.2 Faithfulness of AI-Generated Text

Work on abstractive summarisation has shown that neural models frequently produce content that is not supported by their input. **Maynez, Narayan, Bohnet and McDonald** (ACL, 2020) annotated summaries produced by several models and found a large share of them contained hallucinated content, both contradicting the source and adding unsupported information. **Ji and colleagues** (ACM Computing Surveys, 2023) surveyed hallucination across natural language generation tasks and concluded that it remains an open problem for large language models. **Kasneci and colleagues** (Learning and Individual Differences, 2023) discussed the opportunities and risks of large language models in education and stressed that unchecked errors are especially harmful for learners, who are not in a position to detect them. These findings motivate a central design choice of Learnova: the teacher's text is kept verbatim, and any text written by a model is accepted only after it has been checked against the slide.

2.3 Slide Design and Learning

Cognitive load theory (Sweller, Cognitive Science, 1988) distinguishes intrinsic load, which comes from the material itself, from extraneous load, which comes from the way the material is presented. **Cowan** (Behavioral and Brain Sciences, 2001) argued that working memory holds about four chunks at a time. Slides that present many new elements at once, or force the learner to split attention between separate sources, waste this limited capacity. **Mayer's** principles of multimedia learning (Cambridge University Press, 2009) translate these findings into design rules: coherence (remove material that does not serve the goal), signalling (make the structure explicit), spatial contiguity (place pictures next to the words that describe them) and segmenting (present material in learner-paced parts).

The **assertion–evidence** approach (Alley, Schreiber, Ramsdell and Muffo, Technical Communication, 2006; Garner and Alley, International Journal of Engineering Education, 2013) replaces topic titles and bullet lists with a full-sentence headline that states the slide's main point, supported by visual evidence. Controlled studies found better comprehension and retention with this design. **Tufte** (The Cognitive Style of PowerPoint, 2003) argued that bullet outlines fragment reasoning and hide the relationships between ideas.

2.4 Worked Examples and Interactive Visualisation

Sweller and Cooper (Cognition and Instruction, 1985) showed that students learning algebra benefited more from studying worked examples than from solving the same problems unaided. **Atkinson, Derry, Renkl and Wortham** (Review of Educational Research, 2000) and **Renkl** (Cognitive Science, 2014) refined this into principles: each step should be visible on its own and the reason for the step should be stated. In computing education, the meta-study of **Hundhausen, Douglas and Stasko** (Journal of Visual Languages and Computing, 2002) found that algorithm visualisations help learning when students actively engage with them — changing inputs, predicting results — rather than pas-

sively watching. The **PhET** simulations (Wieman, Adams and Perkins, Science, 2008) and the **ICAP** framework (Chi and Wylie, Educational Psychologist, 2014) support the same conclusion: interactive engagement produces better learning than passive viewing.

2.5 Automatic Visual Design and Layout

Mackinlay's APT (ACM Transactions on Graphics, 1986) chose graphical encodings automatically from the characteristics of data, using explicit expressiveness and effectiveness rules. **Draco** (Moritz and colleagues, IEEE Transactions on Visualization and Computer Graphics, 2019) encoded visualisation design knowledge as constraints that can be inspected and extended. These works inspired the decision engine of Learnova, which applies explicit, inspectable rules to sentences rather than to data tables. For text layout, **Knuth and Plass** (Software: Practice and Experience, 1981) formulated paragraph line breaking as an optimisation over break points solved by dynamic programming; our CLASS algorithm uses the same structure to break a long slide into several slides.

2.6 Commercial Presentation Tools

Gamma, **Microsoft Copilot in PowerPoint**, **Gemini in Google Slides** and **Canva Magic Design** generate a deck by asking a language model for an outline and then filling a template. In our trials with lecture material, all of them produced visually attractive slides from a prompt. When given an existing deck, however, they rewrote and condensed the content, lost figures and equations that existed only as pictures, added no quizzes, solved examples or laboratories, required a subscription for full use and did not work offline.

2.7 Comparison and Research Gap

Table 2: Comparison of existing systems with Learnova.

System	Input is an existing deck	Keeps source content	Explains choices	Illustrations	Interaction	Works offline
PPSGen (2015)	No	No (summarises)	No	No	No	Yes
DOC2PPT (2022)	No	No (abstractive)	No	Paper figures	No	Yes
D2S (2021)	No	No	No	No	No	Yes

System	Input is an existing deck	Keeps source content	Explains choices	Illustrations	Interaction	Works offline
Gamma / Copilot / Gemini / Canva	Partly	No	No	Stock / generic	No	No
Learnova	**Yes**	**Yes (verified)**	**Yes**	**Every slide**	**Quizzes, solvers, labs**	**Yes**

The survey reveals a clear gap. Research systems summarise papers rather than improving lecture decks and trade faithfulness for brevity. Commercial tools produce attractive new decks but cannot be checked against the source, ignore picture-only content and provide no interaction. Research on learning tells us what a good slide looks like and why interaction matters, but no tool applies this knowledge automatically to the decks teachers already use. Learnova is designed to fill exactly this gap.

3. Requirements Analysis and Feasibility

3.1 Stakeholders

The primary users of Learnova are **faculty members**, who upload their existing decks and present or share the redesigned versions, and **students**, who use the web deck, quizzes, explainers and laboratories to study. Secondary stakeholders are the **department**, which benefits from consistent and accessible course material, and **administrators** of the deployment, who manage accounts and API keys.

3.2 Functional Requirements

Table 3: Functional requirements.

ID	Requirement
FR1	The user shall sign in with a mobile number and a one-time password.
FR2	The user shall upload a PPTX, PDF, DOCX, Markdown or text file, or type a topic.

ID	Requirement
FR3	The system shall convert the input into structured slides with titles, bullet levels, tables and figures.
FR4	The system shall show live progress of each processing stage and allow a running job to be cancelled.
FR5	The system shall choose a visual form for every slide and show the reason for the choice.
FR6	The system shall attach an illustration to every topic slide and publish it as a linked HTML page.
FR7	The system shall add checkpoint quizzes, a glossary, key takeaways and an agenda.
FR8	The system shall produce step-by-step explanations for problems it can solve exactly.
FR9	The system shall attach interactive laboratories to suitable topics.
FR10	The user shall download an editable PPTX file, a web deck and a study pack.
FR11	The user shall create and revoke view-only share links.
FR12	The user shall edit slides, save versions and refresh an older deck with the current builders.
FR13	The user shall ask questions about a deck to a tutor that answers only from the deck.
FR14	The user shall add, replace or remove AI provider keys in Settings.

3.3 Non-Functional Requirements

Table 4: Non-functional requirements.

Category	Requirement
Fidelity	At least 98% of the numeric and named facts of the source shall appear in the output.
Layout	No text shall overflow its box and no word shall be broken across lines.
Performance	A 36-page deck shall be processed in under 15 seconds without API keys on a laptop.
Availability	The system shall complete every deck even when every AI provider fails.
Security	Codes shall be stored hashed; requests shall be rate-limited; generated code and SVG shall be sanitised.
Usability	The interface shall work on desktops and phones and be installable as an app.
Maintainability	Every module shall be covered by automated tests that run without API keys.

Category	Requirement

3.4 Hardware and Software Requirements

For development and for running the system for one department, a laptop with a 64-bit processor, 8 GB of memory and 5 GB of free disk space is sufficient. The software stack consists of Python 3.12 with FastAPI, python-pptx, PyMuPDF, python-docx and SymPy; Node.js with React, Vite and Tailwind CSS for the front end; LibreOffice in headless mode for converting PowerPoint files; and the Tesseract OCR engine. Optional API keys for Google Gemini, Groq or NVIDIA enable the AI features. Users need only a modern web browser.

3.5 Feasibility Study

Technical feasibility. Every component of the system relies on mature open-source libraries, and the complete pipeline runs on an ordinary laptop. The prototype is working end to end, with 757 automated tests.

Economic feasibility. Learnova can run entirely offline at no cost. The free tier of Gemini, used through a pool of five keys, is sufficient for AI titles, explanations and diagrams. A department server would cost less per year than a single paid licence of a commercial AI presentation tool.

Operational feasibility. Teachers keep full control: every slide stays editable and every automatic decision is explained. Students need nothing more than a browser, and the web deck also works on phones.

Schedule feasibility. The project was completed in four phases between September 2025 and October 2026: research and design, a first prototype, the Learnova V2 platform, and real-deck support with AI features and research publications.

4. System Design

4.1 Architecture Overview

Learnova follows a layered architecture in which the user interface, the web service, the processing pipeline, the AI services and the storage are separate and replaceable. Figure 1 shows the system as a flow from inputs to outputs. Each layer communicates with the next through a narrow interface: the front end talks only to the REST API, the API starts and monitors jobs, the pipeline converts every input into one intermediate representation, and the builders turn the final slide plan into files.

The five columns of the architecture are:

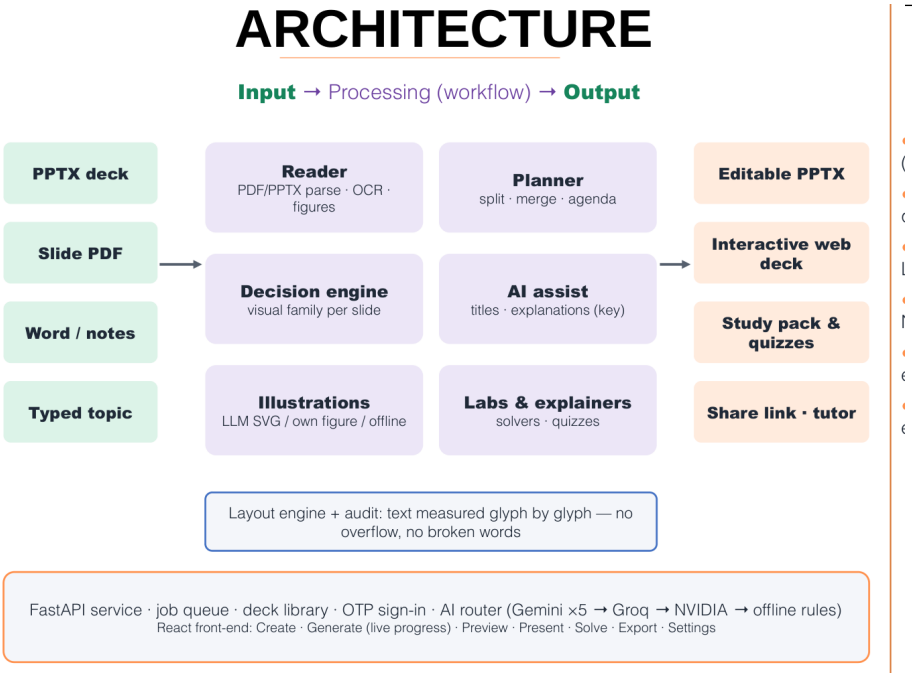


Figure 1: System architecture: inputs, ingestion, engines, AI service and outputs, with deployment and methodology.

1. **Inputs** — a PowerPoint file, a slide PDF, a Word document or notes, a typed topic, and laboratories uploaded by teachers.
2. **Ingestion** — conversion of PowerPoint files to PDF with LibreOffice, the page-by-page slide reader, the OCR decision for picture-only slides, repair rules for page furniture and glued titles, and extraction of the source colour palette.
3. **Engines** — the planner with the CLASS segmentation algorithm, the Content Decision Engine, the layout engine, the exact explainer solvers, and the PSF scorer and audit.
4. **AI service** — the router across Gemini, Groq and NVIDIA providers, AI polishing of titles and explanations, AI-written topic diagrams, AI-written laboratories, and the offline fallbacks.
5. **Outputs** — the editable PowerPoint file, the web deck, the study pack, share links and the presenter console, and the deck tutor.

4.2 Data Flow

A generation request passes through the following stages, each reported to the user with live progress:

1. **Convert.** The uploaded file is converted into Markdown with one section per slide and a metadata record per page (figure, preview image, OCR text, palette). For PowerPoint files, LibreOffice renders a PDF first; the result is cached by a hash of the file so that a repeated upload is instant.
2. **Review.** The user may inspect the extracted outline before generation.

3. **Plan.** Overloaded slides are split with CLASS, very short consecutive parts of one topic are merged, and an agenda, key takeaways, a glossary and checkpoint questions are added.
4. **Polish.** Titles are cleaned and, with a key, rewritten as assertions; thin slides receive a plain explanation and an example.
5. **Decide.** The decision engine chooses a visual family and builds its data.
6. **Illustrate.** Every topic slide receives a picture; diagrams are generated in parallel.
7. **Labs.** Laboratories are matched or generated for suitable topics.
8. **Build.** The PowerPoint and web builders write the files; scores and analysis are computed.
9. **Store.** The deck, its versions and its artefacts are saved in the user's library.

4.3 Module Design

4.3.1 Input parsers

Separate parsers handle each format. The slide-PDF reader (Section 5.1) is the most important, since PowerPoint files are also routed through it. The Word parser keeps headings, lists and tables. Typed topics are expanded into an outline by the language model when a key is present, or used as a single section otherwise.

4.3.2 Content Decision Engine

The engine contains 27 detectors, each responsible for recognising one kind of structure — a process, a cycle, a timeline, a comparison, a definition, a formula, a statistic, a hierarchy, a set relation and others. Each detector returns a family, a confidence, the data needed to draw the visual and a reason. The engine also produces card headings and takeaways. It is described in Section 5.2.

4.3.3 Deck director and builders

The deck director turns the decisions into a slide plan, adds speaker notes, timing estimates and transitions, and ensures that consecutive slides do not repeat the same layout too often. The PowerPoint builder draws every visual with native shapes so that teachers can edit them; the web builder produces a single HTML file using Reveal.js, with click-by-click reveals, flip cards, a matching activity, zoomable figures and an original-slide viewer.

4.3.4 Illustration subsystem

The illustration subsystem chooses the picture for each slide (Section 5.6), generates diagrams with the key pool, validates them, caches them and publishes them at `/api/art/{id}`.

4.3.5 Explainers and laboratories

The explainer subsystem detects solvable problems in slide text and produces step-by-step solutions (Section 5.7). The laboratory subsystem holds built-in laboratories, matches them to slide topics, generates new laboratories and serves them at `/api/labs/{id}` (Section 5.8).

4.3.6 Platform services

The platform provides authentication with one-time passwords, a job queue that survives restarts, a deck library with versions, share links, the study-pack exporter, the presenter console and the deck tutor.

4.4 Database Design

The system stores its records in SQLite and its artefacts on disk. The main tables are listed below.

Table 5: Main database tables.

Table	Key fields	Purpose
users	id, phone, created_at	One row per signed-in user
otp_codes	phone, code_hash, expires_at, attempts	Hashed one-time passwords with expiry and attempt limits
sessions	token_hash, user_id, expires_at	Sign-in sessions
jobs	id, user_id, status, stage, progress, error	Generation jobs, restart-safe
decks	id, user_id, title, version, slide_count, scores	Saved decks with their latest version
deck_versions	deck_id, version, note, created_at	Version history for edits and refreshes
shares	token, deck_id, user_id, revoked	View-only share links

Artefacts (PowerPoint file, web deck, slide payload, images, editable slides) are stored per deck and version. Caches for converted documents, OCR results, language-model responses, diagrams and laboratories are keyed by content hashes.

4.5 API Design

The web service exposes about seventy endpoints. The most important are grouped below.

Table 6: Main API endpoints.

Group	Endpoints
Authentication	POST /api/auth/otp/request, POST /api/auth/otp/verify
Jobs	POST /api/jobs, POST /api/jobs/{id}/generate, GET /api/jobs/{id}, GET /api/jobs/{id}/download/{artifact}
Library	GET /api/decks, GET /api/decks/{id}/editable, PUT /api/decks/{id}/slides, POST /api/decks/{id}/refresh
Sharing	POST /api/decks/{id}/shares, GET /api/share/{token}
Learning	POST /api/explain, GET /api/labs, GET /api/labs/{id}, GET /api/art/{id}, POST /api/decks/{id}/chat
Settings	provider key management and status endpoints

4.6 User Interface Design

The front end is a single-page application with a warm, bright visual style. Its main pages are **Create** (upload or type a topic), **Generate** (live progress of every stage), **Preview** (every slide with its visual, reason, timing and transition, and a full-size preview of the selected slide), **Present** (presenter console with notes, timer and an audience window), **Solve** (type a problem and get a step-by-step explanation), **Export** (downloads, study pack and share links), **Quizzes**, **Gallery** (more than a thousand ready-made lesson decks), **Analytics** and **Settings**. On phones the layout switches to a dedicated mobile design with a bottom tab bar, and the application can be installed as a progressive web app.

5. Methodology and Algorithms

This chapter describes the methods we designed. Each is implemented as a separate module with its own tests.

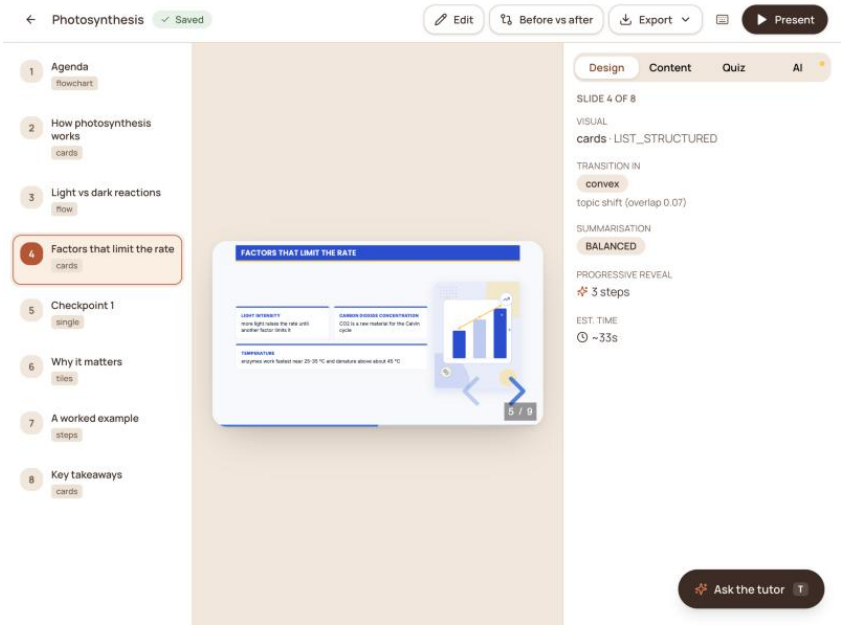


Figure 2: Deck preview page showing each slide, the visual chosen and the reason.

5.1 Recovering the Structure of a Slide Deck

A deck exported to PDF contains one slide per page, but its structure is implicit: bullets are separate glyphs, nesting is only an indent, tables may be pictures and diagrams are drawings. The reader works on text lines, each with a bounding box (x_0, y_0, x_1, y_1), a font size s and a font name.

Page furniture. Course names, author lines and slide numbers repeat on many pages. A line in the top 12% or bottom 14% of a page is treated as furniture when the same normalised text appears in that band on at least

$$m(n) = \max\left(3,\ \lfloor 0.3\, n \rfloor\right)$$

of the n pages. Pictures that repeat on many pages (logos, backgrounds) are removed in the same way, but counted only on pages where they are actually drawn, because some exporters list every image of the deck in a shared resource dictionary.

Titles. With $\bar{s}$ the body font size (the size that carries the most characters), the largest line ℓ^* in the top third of a page of height H is accepted as the title when

$$s(\ell^*) \geq 1.18\, \bar{s} \quad \text{or} \quad \left(\text{bold}(\ell^*) \text{ and } y_0(\ell^*) < 0.2H\right).$$

A following line continues a wrapped title when it has the same size, starts at the same horizontal position and its vertical gap g satisfies $-0.5\, s < g < 0.9\, s$.

Bullet levels. The horizontal positions of bullet glyphs are clustered with a tolerance of 9 points into anchors $a_1 < a_2 < \cdots < a_K$, and an item that starts at position x receives the level $\ell(x) = \arg\min_k |x - a_k|$. Lines without a glyph are joined to the previous item when they continue an unfinished sentence.

Glued titles. Titles written in capitals sometimes lose their spaces ("CONSTRUCTINGTH-ETABLE"). Such a token t is segmented by dynamic programming over a vocabulary V built from the deck itself and a dictionary:

$$B(j) = \max_{i<j,\ t_{i:j}\in V} \big(B(i) + (j-i)^2\big), \qquad B(0) = 0.$$

Squaring the length of each word favours a few long words over many short ones.

Picture-only slides. When the largest picture covers at least 45% of the page and the page has at most twelve words of text, the picture is read with Tesseract. Let I be the set of dark pixels and M the area covered by recognised words; the *graphic ratio*

$$\gamma = \frac{|I \setminus M|}{|I|}$$

is close to zero for a screenshot of text and large for a diagram. The recognised text replaces the picture only when the mean confidence is at least 62, there are at least twelve words, $\gamma \le 0.42$ and at most 15% of the lines are symbol debris; otherwise the picture is kept as a figure. This rule matters most for equations: a derivation read by OCR as random symbols is worse than the picture of the derivation.

Charts and continuation sentences. When most "bullets" of a slide are numbers of fewer than four words, they are the axis ticks of a drawn chart; the chart region is rendered as an image instead. A bullet that begins with "means", "is" or "refers to" is joined to the label before it, so "Low variance:" followed by "means that the model…" becomes one complete sentence.

5.2 Content Decision Engine

For each slide, the 27 detectors are run. A detector returns either nothing or a tuple (f, v, c, D, r): the visual family f, a variant v, a confidence $c \in [0, 1]$, the data D the renderer needs and a reason r in plain words. Tables, section outlines and reference lists are tested first. The remaining candidates are filtered by exclusion rules learned from the gold set — for example, a cycle removes a linear process and a definition removes statistic tiles — and the decision is

$$d^* = \arg\max_{d \in X(\mathcal{C})} c_d.$$

The three next-best candidates are kept as alternatives for the slide editor. If no detector fires, the slide

is shown as key-point cards that carry every sentence in full, so a visual choice never removes text. Card headings are formed from the sentence itself: discourse cues and pronouns are removed, the first three content words are kept, the heading is cut at an unbalanced parenthesis, and trailing function words, modal verbs and adverbs are removed so that a heading never ends in the middle of a phrase.

5.3 Pedagogical Slide Fitness (PSF)

PSF is a metric we designed to estimate how suitable a slide is for learning. It combines three components, each between 0 and 1.

Information efficiency rewards a slide for introducing new concepts without repeating itself. With $\mathcal{N}(s)$ the concepts on slide s that did not appear on earlier slides, a rarity weight $\text{idf}(w)$ between 0.4 and 3.0 that grows with word length, and the redundancy $\rho(s)$ measured as the mean overlap of each bullet with the title or the other bullets,

$$U(s) = \sum_{w \in \mathcal{N}(s)} \text{idf}(w), \qquad E(s) = \frac{U(s)}{U(s) + \kappa_e}\,(1 - \rho(s)).$$

Cognitive load estimates how much working memory the slide demands. With k distinct concepts, relational-cue density r, word count t, the largest number of visual elements v and a split-attention indicator σ,

$$L(s) = \text{sigmoid}(w_e \hat{e} + w_t \hat{t} + w_v \hat{v} + w_\sigma \sigma - \theta),$$

where $\hat{e} = \min(1, k(1+r)/e_{\text{ref}})$, $\hat{t} = \min(1, t/t_{\text{ref}})$ and $\hat{v} = \min(1, v/v_{\text{ref}})$.

Multimedia coherence $C(s)$ is the mean of four indicators taken from Mayer's principles: signalling (a real title and a takeaway), coherence (the share of sentences on the slide's topic), non-redundancy (a picture's description does not repeat the text) and spatial contiguity (the figure is on the same slide as its explanation).

The slide score is the weighted geometric product

$$\text{PSF}(s) = E(s)^{\alpha}\,(1 - L(s))^{\beta}\,C(s)^{\gamma}, \qquad \alpha + \beta + \gamma = 1,$$

so that a very high load cannot be compensated by more information. For a deck, the geometric mean of the slide scores is multiplied by a flow term that rewards a moderate overlap of concepts between consecutive slides. The default parameters are listed below.

Table 7: PSF and CLASS parameters.

Parameter	Meaning	Value
α, β, γ	weights of E, $1-L$ and C	0.40, 0.42, 0.18
κ_e	half-saturation of new information	6
w_e, w_t, w_v, w_σ	load weights	1.10, 1.35, 0.85, 1.60
θ	load bias	2.30
$e_{\text{ref}}, t_{\text{ref}}, v_{\text{ref}}$	load references	9, 16, 8
λ	cost of one more slide	0.35
μ	weight of missing information	0.9
ν	overflow penalty	0.25

5.4 CLASS: Load-Optimal Slide Segmentation

When a slide has too many points, it must be split into several slides. A naive method splits the points evenly. CLASS (Cognitive-Load-Aware Slide Segmentation) instead chooses the split that minimises the total cognitive cost. Given points $b_1, \dots, b_m$, a maximum of P points per slide and a soft target T, CLASS chooses break points that minimise

$$J = \sum_{k=1}^{K} \Big[L(G_k) + \mu(1 - E(G_k)) + \nu \max\left(0, |G_k| - T\right)^2 \Big] + \lambda K,$$

where G_k is the k-th group of consecutive points. Because the cost of a group depends only on its own points, the optimum is found by the recurrence

$$F(i) = \min_{\max(0, i-P) \le p < i} \big[F(p) + \text{cost}(b_{p+1..i}) + \lambda \big], \qquad F(0) = 0,$$

in $O(mP)$ time. Every point is placed exactly once, so segmentation never loses content.

5.5 Layout Without Broken Words

The most visible defect of automatically generated slides is a word split across two lines. Learnova prevents it with two width models. For counting lines, a box W inches wide holds about $\lfloor 72W/(\bar{\rho}\, s) \rfloor$ characters per line at font size s, with $\bar{\rho} = 0.52$ for regular and 0.575 for bold text. For single words,

the width is measured character by character from the font metrics:

$$w(u, s) = \kappa \frac{s}{72} \sum_{ch \in u} \rho(ch),$$

with a safety factor $\kappa = 1.06$. The font size of a box is the largest size on a half-point grid at which the text fits, never below 11 points for body text; content that still does not fit moves to a continuation slide. Finally every paragraph is scaled by

$$\sigma = \max\Big(0.62,\ \min\big(1,\ \min_{u} W / w(u, s)\big)\Big),$$

which guarantees that the longest word fits on one line. Wide visuals such as process flows, timelines and tables keep the full width of the slide.

5.6 Illustrations with Two Fallbacks

Every topic slide receives a picture, chosen in this order: the source's own figure, shown beside the text; a diagram written by a language model; and a scene from the offline illustrator. For the AI diagram, the model receives the slide's title and text and is asked for one SVG figure that shows the mechanism of the idea, labelled with the slide's own terms, in the deck's colours. The answer is sanitised before use: scripts, embedded images, external links, filters and style sheets are removed, inline styles are converted into attributes, and the diagram must contain at least five shapes and render correctly. Labels that would run past the edge are scaled down using the width model of the previous section. Accepted diagrams are cached by a hash of the slide text, embedded in the web deck, rasterised for PowerPoint and published as an HTML page to which the PowerPoint picture links. Offline scenes are published and linked in the same way, so every picture in the PowerPoint opens its own page.

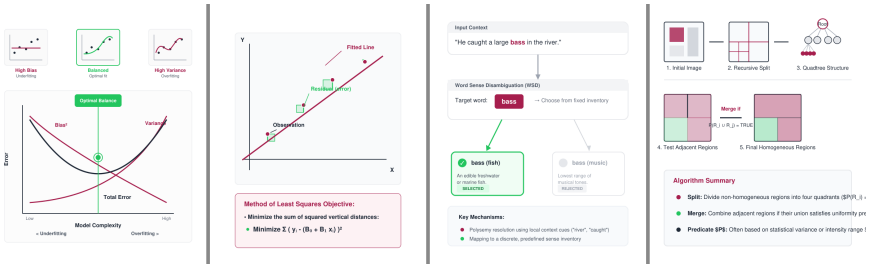


Figure 3: Diagrams written by the language model for four slides.

Key-pool scheduling. The free tier of the Gemini API allows a limited number of requests per minute per key. Learnova uses a pool of keys: each key is paced separately with a minimum interval $\Delta = 4.2$ seconds, two requests are kept in flight per key, and a key that reports an exhausted quota or an invalid credential is skipped for fifteen minutes. With $|K|$ keys the sustained rate is $|K|/\Delta$ requests per

second. When every key fails, the system continues with offline logic, so a deck is never blocked by an external service.

5.7 Step-by-Step Explainers

When a slide contains a problem that the system can solve exactly, it produces an explainer: a sequence of steps, each with a short action title, the working with the changed part highlighted, a plain-language reason and the general rule behind it. Candidate problem statements are taken from the whole slide, from pairs of neighbouring equations, from each line and from each sentence; each candidate is offered to the solvers in a fixed order and the first accepted solution is used. An explainer is accepted only if it has at least three steps and every step has a reason. All arithmetic is exact, using symbolic algebra and fractions, so a number shown to a student is never guessed. With a key, a problem that no solver recognises may be explained by the language model; such an explanation is clearly labelled as not computed by the system.

5.8 Interactive Laboratories

All laboratories share one page structure: preset examples, a view of the current state, an explanation panel with *Step*, *Back* and *Play* controls, and a form for the student's own input. A laboratory is defined by a function that turns an input into a list of frames, each frame being one small decision with its explanation. A slide receives a built-in laboratory when its title matches the laboratory's topics. Otherwise a laboratory may be generated under a policy that keeps decks usable: one laboratory per topic, the same laboratory at most twice per deck, at most ten generated laboratories per deck, and none for agenda, summary or quiz slides or for titles that look like OCR noise. With a key, the language model writes the laboratory's code, which must pass a syntax check and may not use network or storage functions; without a key, the laboratory is built from the slide's own structure — a process is walked step by step, a comparison is explored criterion by criterion, and concepts are revealed one at a time after a "think first" prompt.

5.9 Grounded Language-Model Assistance

With a key, the system asks the language model for an assertion title, a short explanation for thin slides, an example and up to two key terms per slide, in batches of ten slides. Each answer is checked before use: a title must be a full sentence of four to fifteen words, must share at least one content word with the slide and must not begin with a command; an explanation must share at least two content words with the slide. Answers that fail are discarded and the rule-based version is kept.

5.10 Checkpoint Quizzes

Quizzes are generated without a key from the decisions of the engine: definitions become "which term matches this description" questions, ordered processes become "what comes next" questions when their steps are clean, comparison tables become look-up questions, and statistics become "what was the figure for" questions. Distractors are drawn from the same deck and matched in form to the correct answer. Questions built from slide-part counters, symbol debris or example sentences are rejected.

6. Implementation

6.1 Technology Stack

Table 8: Technology stack.

Layer	Technologies	Role
Front end	React 18, Vite, Tailwind CSS	Single-page application, responsive and installable
Web deck	Reveal.js, inline SVG, vanilla JavaScript	Interactive presentation with reveals, overlays and labs
Back end	Python 3.12, FastAPI, Pydantic	REST API, background jobs, authentication
Storage	SQLite, file store, content-hash caches	Users, jobs, decks, versions, artefacts
Documents	python-pptx, PyMuPDF, python-docx, LibreOffice	Reading inputs and writing PowerPoint files
OCR	Tesseract	Reading picture-only slides
AI	Google Gemini (five-key pool), Groq, NVIDIA NIM	Titles, explanations, diagrams, laboratories
Mathematics	SymPy, exact fractions	Step-by-step solvers
Quality	pytest, layout audit, gold-set evaluation	757 automated tests

6.2 Project Structure

The code is organised as a reusable library (`src/learnova`) used by the web service (`apps/api`) and by command-line scripts (`scripts`). The library is divided into packages: `parsers` (input

readers, including the slide-PDF reader), `pipeline` (orchestrator, planner, enrichment, polishing, density control), `visuals` (the Content Decision Engine and icons), `rendering` (deck director, PowerPoint builder, web builder, layout engine), `ai` (assistance, topic diagrams, scene illustrator, quizzes), `explain` (problem detection and solvers), `labs` (laboratory engine, built-in and generated laboratories), `providers` (language-model providers and the router), `scoring` (PSF and CLASS), `evaluation` (layout audit and visual accuracy), `storage` (deck library, versions, shares) and `export` (study pack). The front end lives in `frontend/src` with one component per page.

6.3 Key Modules

Orchestrator. The orchestrator runs the pipeline as named stages — convert, plan, polish, decide, illustrate, labs, score, build — through a stage runner that records the duration and outcome of each stage, reports progress to the job record and isolates failures, so that a failing optional stage (for example, illustration) never stops the deck from being built.

Slide reader. The reader implements the rules of Section 5.1 on top of PyMuPDF. It first converts PowerPoint files to PDF with a private LibreOffice profile, so that an instance of LibreOffice open on the desktop does not block conversion, and caches the result by file hash.

Decision engine. Each of the 27 detectors is a small function. The engine runs them in a fixed order, applies the exclusion rules and returns a decision object that carries the family, data, confidence, reason and alternatives.

Builders. The PowerPoint builder draws every visual family with native shapes and text boxes sized by the layout engine. Pictures are placed beside the content and carry hyperlinks to their HTML pages; laboratory buttons link to `/api/labs/{id}`. The web builder writes a single self-contained HTML file in which illustrations are live SVG, laboratories open in an overlay, and the original slide can be shown next to the redesigned one.

Topic diagrams. The diagram module sends one request per topic slide through the key pool, sanitises the answer, renders it with MuPDF to make sure it is valid, and caches it. Diagrams are generated in parallel with two requests in flight per key.

Router. The language-model router keeps a preference order per task (for example, fast models for short classification tasks), adapts model names when it falls through to another provider, and switches a provider off for fifteen minutes after repeated failures or a single invalid-key error.

6.4 User Interface

The Create page accepts a file by drag and drop or a typed topic, and offers options for the slide density, the design and whether quizzes and illustrations should be added. During generation, the Generate page shows each stage with its status. The Preview page lists every slide with its chosen visual and the reason, and shows the selected slide at full size; slides can be edited and saved as new

versions. The Solve page accepts a problem and shows the step-by-step explanation, which can be downloaded as a page. The Export page provides the PowerPoint file, the web deck, the study pack and share links.

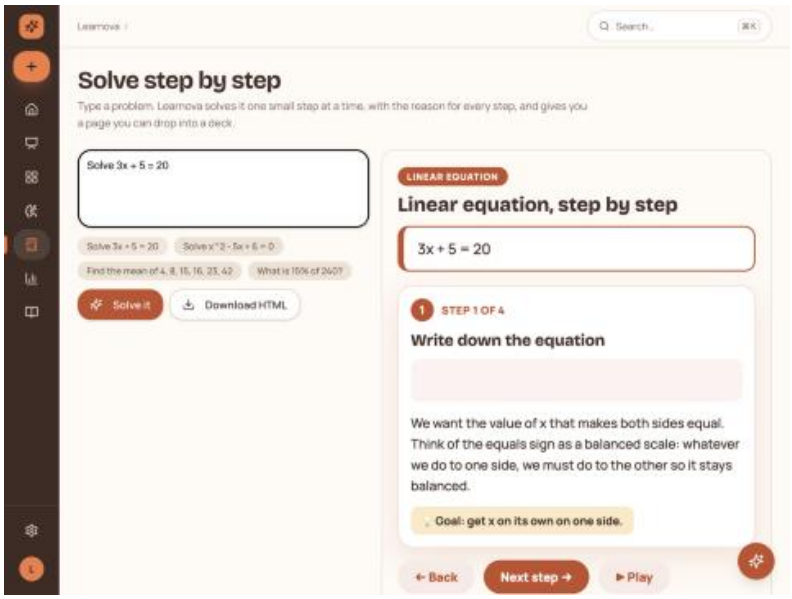


Figure 4: Solve page: a problem explained step by step.

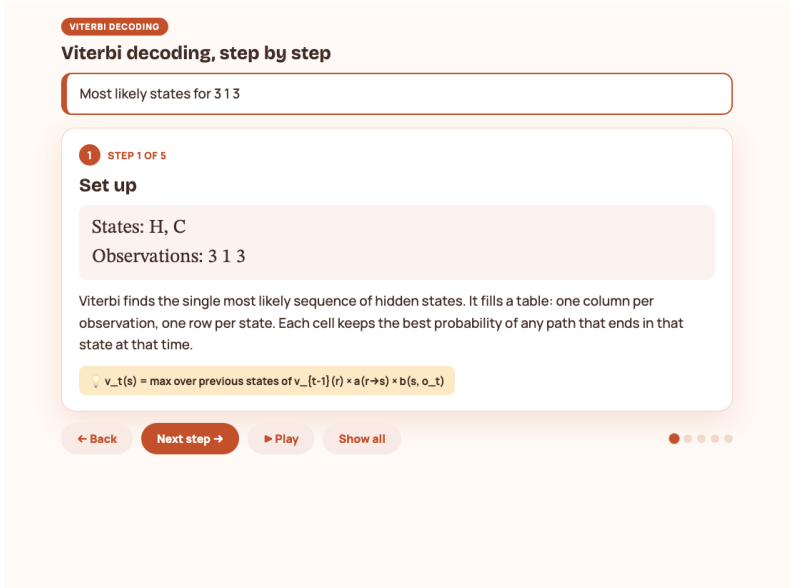


Figure 5: Viterbi explainer: every value computed exactly and explained.

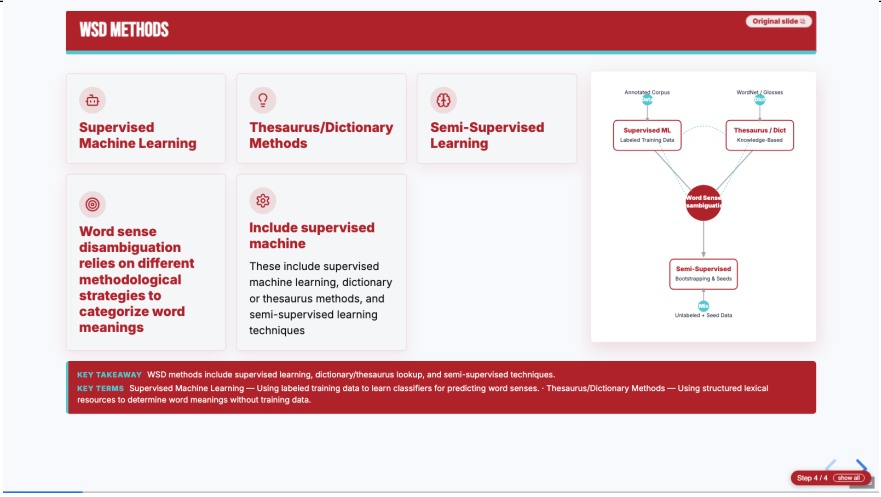


Figure 6: Interactive web deck: an AI-drawn diagram beside the slide content.

6.5 Testing

The project has 757 automated tests that run in about 25 seconds. They cover each parser, each detector of the decision engine, the layout guarantees (no broken words, no overflow), the quiz rules, every solver, the laboratory policy, the diagram sanitiser, the key pool, the API endpoints and complete end-to-end generation. The tests always run without AI keys: the test configuration blanks every key, so that a key in the developer's environment cannot change the results. In addition, an end-to-end script drives the running API with real files, a layout-audit tool checks finished PowerPoint files, and the gold sets measure the accuracy of the decision engine.

6.6 Improvements after testing on the review decks

Before the final review we rebuilt the teachers' own decks with the current code and inspected every page as a panel member would. This round found problems that the automated tests had not covered, and each was fixed in the code, given a regression test where possible, and pushed to the repository as a separate change. The table below summarises them.

Table 9: Problems found on real decks and how they were fixed.

Problem seen on the rebuilt deck	Cause	Change made	Effect
OCR scraps of pictures ("M=0 1 ee", axis ticks) appeared as numbered points	text read from a figure was appended to the slide text	figure text is kept beside the slide and used only as a hint	no OCR debris on any slide of the test decks
A teacher's build-up slides (same title, one more row each time) appeared three times	each exported page became a slide	a page whose words are 90% contained in the next page with the same title is dropped	"Advantages (1/3)… (3/3)" became one complete slide
Algorithms and runs of equations were split into separate numbered cards	the generic card layout was chosen	new ALGORITHM visual: one panel, numbered lines, indentation kept, keywords highlighted, equations centred	pseudocode and derivations read top to bottom
The plain-words explanation sat in a 10 pt footer band	the footer carried all secondary text	an "In plain words" panel in the body, sized to its text at 11–14 pt, at most 40% of the area	explanations readable from the back of the room
Captions under figures showed unreadable OCR	OCR output was always used as the caption	a caption is shown only when most of its words are dictionary words	no garbled captions
AI diagrams rendered with black boxes	the PDF library cannot paint SVG gradients	diagrams rendered with resvg; gradients flattened when it is missing	diagrams look as designed
Equations appeared as plain text ("x^2", "(1/2m)")	the slide text was copied as typed	equations parsed into fractions, powers, sums and roots and written as native Office Math (PPTX) and MathML (web)	equations look as they do in Word

Problem seen on the rebuilt deck	Cause	Change made	Effect
Pseudocode lost its indentation	export flattens bullet levels	a depth counter re-indents after for/while/if/repeat and dedents at end/until/else	loops and branches read correctly
Long decks had a quiz only every nine slides	spacing grew with deck length	a checkpoint every four content slides, at most twenty per deck	students answer something every few minutes

The illustrations were also redesigned. Each slide is first classified into the kind of figure that would teach it — a *flow* for processes and apparatus, a *plot* for quantitative relations, an *anatomy* for parts and types, a *comparison*, or a concrete *scene* — and slides where a drawing adds nothing (agendas, references, quizzes, tables, algorithms, slides that are already figures) get none. The prompt asks for six to fourteen labels, the slide's own numbers and three to six numbered parts, each grouped with a short title. The linked HTML page uses those groups to step through the figure part by part, and a part can be highlighted by pointing at it, so the illustration is something a student explores rather than a static picture. Appendix A shows the Haber process plant drawn this way: the gas sources, the compressor at 200 atmospheres, the reactor with its iron catalyst at 450 °C, the condenser and the recycle loop, each numbered in teaching order.

Two further gaps were closed. Problem slides that none of the sixteen exact solvers recognises now receive a step-by-step explanation written by the language model, at most six per deck, and the page states that these steps were not computed by Learnova. Second, the project can now be deployed: a Dockerfile builds one container with the API, the web application, LibreOffice, Tesseract and an English dictionary; a Render blueprint creates the service, and a Vercel configuration forwards the web application's API calls when the two are hosted separately.

7. Results and Evaluation

7.1 Evaluation Setup

We evaluated Learnova on two kinds of material. The **gold sets** are slide texts labelled by hand with the set of acceptable visual families: a development set of 38 slides, a held-out set of 32 slides and a blind set of 31 slides written by a team member who had not seen the rules. The **real decks** are lecture decks from our own courses: two machine-learning PowerPoint files (28 and 63 slides), four slide PDFs (36, 44, 95 and 216 pages) and two Word documents of notes. All timings were measured on a laptop.

7.2 Visual Decision Accuracy

A slide counts as correct when the chosen family is one of the acceptable families, it is drawn as a real visual rather than plain text, and the renderer receives the data it needs.

Table 10: Visual decision accuracy on the gold sets.

Gold set	Slides	Strict accuracy	Drawn as a visual	Families used
Development	38	97.4%	100%	24
Held-out	32	96.9%	100%	21
Blind	31	96.8%	100%	21

Each set had a single miss, in every case a slide for which two families were defensible and the annotator had accepted only one. The median time for a decision was 0.48 milliseconds (95th percentile 0.82 milliseconds), so the engine adds no noticeable time to a deck.

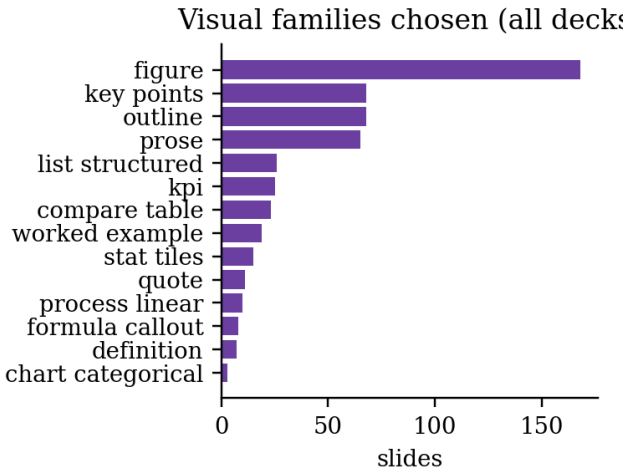


Figure 7: Visual families chosen over the rebuilt decks.

7.3 Fidelity, Layout and Speed

Fact retention is the share of facts in the source — numbers with units, years, percentages and capitalised terms — that also appear in the generated deck:

$$R = \frac{|F_{\text{src}} \cap F_{\text{deck}}|}{|F_{\text{src}}|}.$$

Table 11: Rebuilt decks without an API key.

Deck	Source pages	Slides	Time (s)	Fact retention	Slides flagged
ML: Bias and variance (PPTX)	28	33	11.7	100%	6%
ML: Linear regression (PPTX)	63	69	44.0	100%	7%
NLP Module 4 (PDF)	36	47	9.8	100%	0%
WSD lecture slides (PDF)	44	53	14.8	98%	4%
NLP Module 5 (PDF)	95	100	21.2	99%	3%
NLP Syntax analysis (PDF)	216	225	34.4	100%	3%

No slide in any deck was flagged for overflowing text or for a broken word. All flags marked slides that are sparse because their source contained a single short line, which Learnova does not pad with invented content. Run time grows roughly linearly with the length of the deck; PowerPoint files take longer per page because they are converted to PDF first and contain many full-page figures.

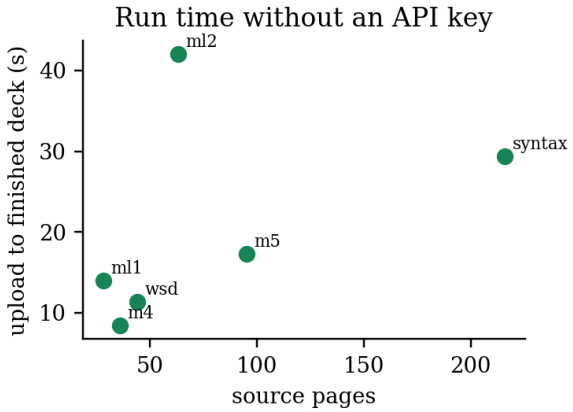


Figure 8: Time from upload to finished files against source length.

With API keys, the same 36-page deck took about 74 seconds, most of which was spent writing laboratories and diagrams; generating laboratories in parallel reduced this from 163 seconds. When every key was rejected, the deck was still completed in 28 seconds because dead keys are switched off at once.

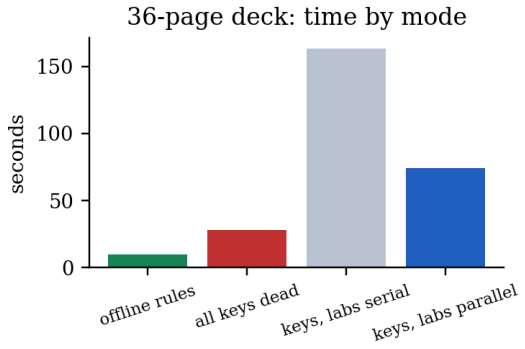


Figure 9: Time for a 36-page deck in different modes.

7.4 Before and After

The most direct measure of the project's value is how much each deck changes. The table compares each source deck with the deck Learnova built from it with keys.

Table 12: Source decks against rebuilt decks.

Deck	Untitled slides (source)	Slides with a picture (source → rebuilt)	Visual slides	Quizzes	Lab links	Linked illustrations	Fact retention
ML: Bias and variance	14	22 → 29	22	5	2	12	100%
ML: Linear regression	36	49 → 64	45	4	8	23	100%
WSD lecture slides	1	7 → 46	52	7	10	40	98%

Deck	Untitled slides (source)	Slides with a picture (source → rebuilt)	Visual slides	Quizzes	Lab links	Linked illustrations	Fact retention
NLP Module 4	0	13 → 43	43	6	3	37	100%
NLP Module 5	32	49 → 96	89	6	13	78	99%

Across the five decks, untitled slides fell from 83 to zero; 251 slides were drawn as structured visuals and 190 illustrations were linked to their own pages, while none of the sources had any visual structure, quiz or laboratory. The PowerPoint route made a large difference: on the linear-regression deck the earlier shape-by-shape parser kept only 23 of 63 slides and produced titles such as "Slide 9", whereas the new route kept every slide and showed the original derivations as figures.

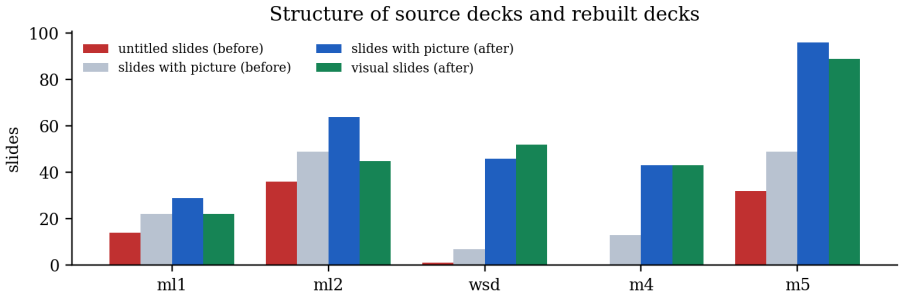


Figure 10: Structure of the source decks and the rebuilt decks.

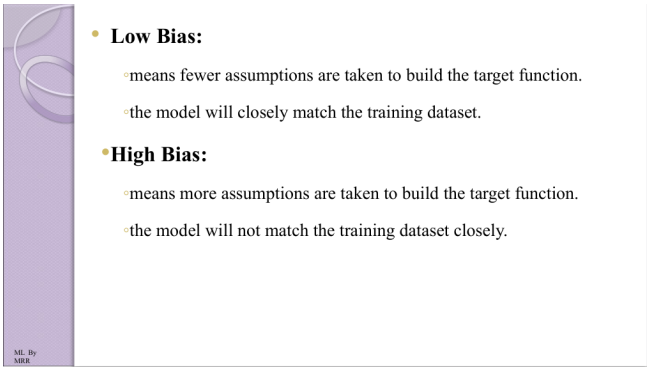


Figure 11: Before: an original slide.

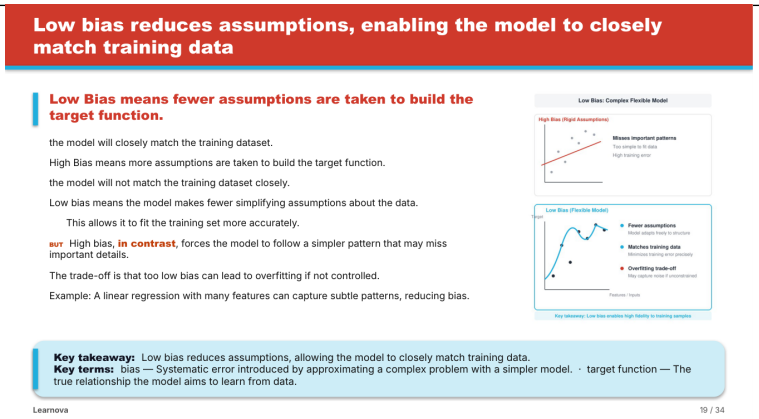


Figure 12: After: the same slide rebuilt by Learnova, with an assertion title, full text, the teacher's figure and key terms.

7.5 PSF and CLASS

We computed PSF for each source deck read as it is and for its rebuilt deck. Coherence rose on five of six decks, because every slide gains a real title, a takeaway and a picture next to its text. The mean PSF rose clearly on the two longest decks (from 0.37 to 0.49 and from 0.15 to 0.24) and was unchanged on one, but fell on the two machine-learning decks and the WSD deck. The reason is visible in the components: on those decks the rebuilt slides carry more words, because explanations, examples and key terms are added to thin slides; this raises the estimated load and, because the added sentences repeat concepts already introduced, lowers the information efficiency. In other words, PSF penalises exactly the enrichment we add for readability. This is an honest limitation of a metric built from theory alone, and calibrating its weights against student ratings is part of our future work.

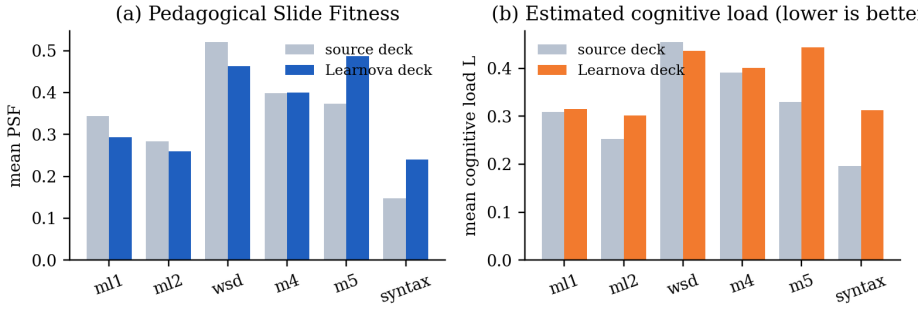


Figure 13: Mean PSF and mean estimated cognitive load of each source deck and its rebuilt deck.

For CLASS, every source slide with more than six points (75 sections) was split with a limit of five points per slide, once with CLASS and once evenly. Both methods used 179 slides in total. CLASS had a lower total cost on 28 sections (37%) and an equal cost on the rest, and it was never worse, as expected of an exact optimiser.

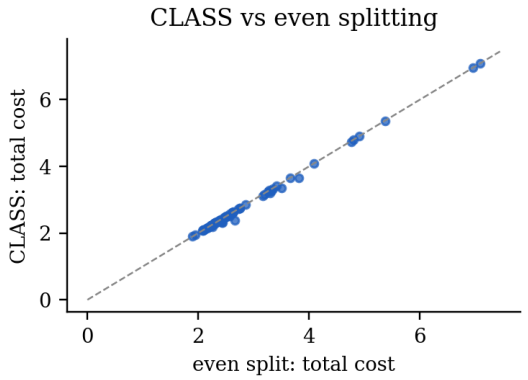


Figure 14: Total cost per overloaded section, CLASS against even splitting.

7.6 Step-by-Step Explainers

We wrote 26 problems that do not appear in the tests, with new numbers and new wordings across all solver types, and worked out each answer by hand. The solvers detected 25 of them on the first run; the missed phrasing (“Find x if 4x + 9 = x + 21”) was added, after which all 26 were detected, classified and answered correctly, and every step carried a reason. None of 52 ordinary sentences (12 from lecture notes and 40 written to resemble problems) was mistaken for a problem. The median time per problem was 0.7 milliseconds. During this exercise two of our own hand-worked answers turned out to be wrong, and the solvers were right in both cases.

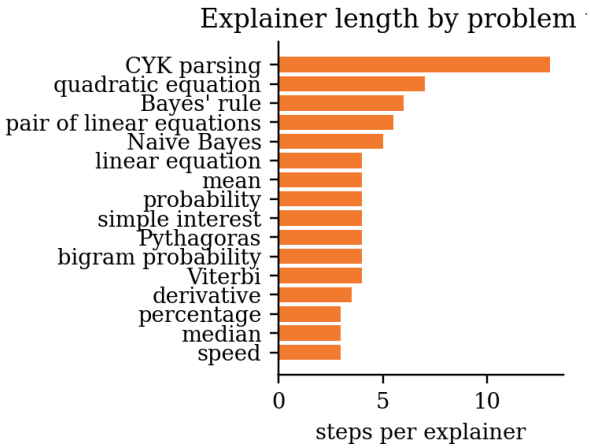


Figure 15: Average explainer length by problem type.

7.7 Illustrations

Without a key, every topic slide receives an offline illustration. With the five-key pool, ten diagrams were written and accepted in 10.1 seconds in parallel. Before we replaced outright rejection with

sanitisation, only 71 of 206 model answers (34.5%) passed validation, most of them failing because of inline styles and gradients; after the change all diagrams of a live check were accepted.

8. How Learnova Solves the Problem Differently

It redesigns the deck the teacher already has. Every existing tool we studied starts from a prompt or a paper and produces a new deck. Learnova starts from the lecture deck itself, so the teacher's structure, wording, examples and figures remain, and the result can be compared slide by slide with the original.

It is faithful by construction. Source text is kept verbatim. Text written by a language model is used only after it is checked against the slide's own words, numbers in solved problems are computed rather than generated, and fact retention is measured on every deck. On real decks, 98–100% of the facts were retained.

It reads real lecture material. Picture-only slides, symbol-font bullets, titles that lost their spaces, repeated logos, overlapping text boxes and PowerPoint files with untitled slides are all handled by explicit rules. The decision to use OCR is itself a rule with a measurable threshold, so a derivation is never replaced by unreadable text.

Every slide is visual, and every choice is explained. The decision engine chooses a visual for every slide with a reason and alternatives, and every topic slide carries an illustration that opens as its own page — a feature none of the compared tools offers.

It turns reading into practice. Checkpoint quizzes, exact step-by-step explainers and interactive laboratories follow the research on worked examples and active learning.

It has its own quality model. PSF gives a theory-based estimate of how suitable each slide is for learning, and CLASS uses that estimate to split long slides optimally. Both are original contributions of this project.

It never depends on a paid service. With free API keys it writes better titles, explanations and diagrams; without them, or when quotas run out, it finishes the deck with its own logic. The whole system runs on a laptop.

Its layout is guaranteed. Text is measured glyph by glyph, so nothing overflows and no word is ever split across lines.

9. Conclusion and Future Scope

9.1 Conclusion

Learnova shows that the decks teachers already use can be turned into clear, illustrated and interactive lessons automatically without losing what they say. By treating the source content as fixed and making

every structural decision with an explicit rule, the system kept 98–100% of the facts of real course decks, removed every untitled slide, chose an appropriate visual for about 97% of slides and never broke a word across lines. Language models are used only where their output can be checked — titles, explanations, diagrams and laboratories — and the system continues to work when every key is exhausted. Two original methods, the PSF metric and the CLASS segmentation algorithm, connect the design of the system to research on cognitive load and multimedia learning. The work has been written up as two research papers, one in IEEE conference format and one in Springer LNCS format.

9.2 Limitations

The decision engine covers the structures we have written detectors for; slides whose meaning lies mainly in a picture still depend on the original figure. OCR of photographed equations is unreliable, which is why such pictures are kept rather than transcribed. Without an API key, slides that are thin in the source remain thin, because the system does not invent content. PSF's weights are taken from the literature and have not yet been calibrated against student judgements, and we have not yet run a controlled study of learning outcomes.

9.3 Future Scope

1. A classroom study with our own batch to measure the effect of the redesigned decks on learning, and calibration of PSF against student ratings.
2. Narrated video export of every deck using text-to-speech and the generated speaker notes.
3. More built-in laboratories for machine learning and image processing topics, and a library where teachers can share their laboratories.
4. A teacher review mode in which each AI suggestion can be accepted or rejected slide by slide.
5. Decks in Hindi and regional languages.
6. Integration with learning management systems such as Moodle and Google Classroom.
7. Deployment as a department service: the front end on a static host and the back end in a container with LibreOffice and Tesseract on a server with persistent storage.

References

1. Y. Hu and X. Wan, "PPSGen: Learning-based presentation slides generation for academic papers," *IEEE Transactions on Knowledge and Data Engineering*, vol. 27, no. 4, pp. 1085–1097, 2015.
2. T.-J. Fu, W. Y. Wang, D. McDuff and Y. Song, "DOC2PPT: Automatic presentation slides generation from scientific documents," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, pp. 634–642, 2022.
3. E. Sun, Y. Hou, D. Wang, Y. Zhang and N. X. R. Wang, "D2S: Document-to-slide generation

via query-based text summarization," in *Proceedings of NAACL*, pp. 1405–1418, 2021.

4. J. Maynez, S. Narayan, B. Bohnet and R. McDonald, "On faithfulness and factuality in abstractive summarization," in *Proceedings of ACL*, pp. 1906–1919, 2020.
5. Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
6. E. Kasneci et al., "ChatGPT for good? On opportunities and challenges of large language models for education," *Learning and Individual Differences*, vol. 103, 102274, 2023.
7. J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988.
8. N. Cowan, "The magical number 4 in short-term memory," *Behavioral and Brain Sciences*, vol. 24, no. 1, pp. 87–114, 2001.
9. R. E. Mayer, *Multimedia Learning*, 2nd ed. Cambridge University Press, 2009.
10. M. Alley, M. Schreiber, K. Ramsdell and J. Muffo, "How the design of headlines in presentation slides affects audience retention," *Technical Communication*, vol. 53, no. 2, pp. 225–234, 2006.
11. J. K. Garner and M. P. Alley, "How the design of presentation slides affects audience comprehension: A case for the assertion–evidence approach," *International Journal of Engineering Education*, vol. 29, no. 6, pp. 1564–1579, 2013.
12. E. R. Tufte, *The Cognitive Style of PowerPoint*. Graphics Press, 2003.
13. J. Sweller and G. A. Cooper, "The use of worked examples as a substitute for problem solving in learning algebra," *Cognition and Instruction*, vol. 2, no. 1, pp. 59–89, 1985.
14. R. K. Atkinson, S. J. Derry, A. Renkl and D. Wortham, "Learning from examples: Instructional principles from the worked examples research," *Review of Educational Research*, vol. 70, no. 2, pp. 181–214, 2000.
15. A. Renkl, "Toward an instructionally oriented theory of example-based learning," *Cognitive Science*, vol. 38, no. 1, pp. 1–37, 2014.
16. C. D. Hundhausen, S. A. Douglas and J. T. Stasko, "A meta-study of algorithm visualization effectiveness," *Journal of Visual Languages and Computing*, vol. 13, no. 3, pp. 259–290, 2002.
17. C. E. Wieman, W. K. Adams and K. K. Perkins, "PhET: Simulations that enhance learning," *Science*, vol. 322, no. 5902, pp. 682–683, 2008.
18. M. T. H. Chi and R. Wylie, "The ICAP framework: Linking cognitive engagement to active learning outcomes," *Educational Psychologist*, vol. 49, no. 4, pp. 219–243, 2014.
19. J. Mackinlay, "Automating the design of graphical presentations of relational information," *ACM Transactions on Graphics*, vol. 5, no. 2, pp. 110–141, 1986.
20. D. Moritz et al., "Formalizing visualization design knowledge as constraints: Actionable and extensible models in Draco," *IEEE Transactions on Visualization and Computer Graphics*, vol. 25, no. 1, pp. 438–448, 2019.
21. D. E. Knuth and M. F. Plass, "Breaking paragraphs into lines," *Software: Practice and Experience*, vol. 11, no. 11, pp. 1119–1184, 1981.
22. R. Smith, "An overview of the Tesseract OCR engine," in *Proceedings of ICDAR*, pp. 629–633,

2007.
23. A. Meurer et al., "SymPy: Symbolic computing in Python," *PeerJ Computer Science*, vol. 3, e103, 2017.
24. H. El Hattab, "reveal.js: The HTML presentation framework," https://revealjs.com, accessed October 2026.

Appendix A: Sample Outputs

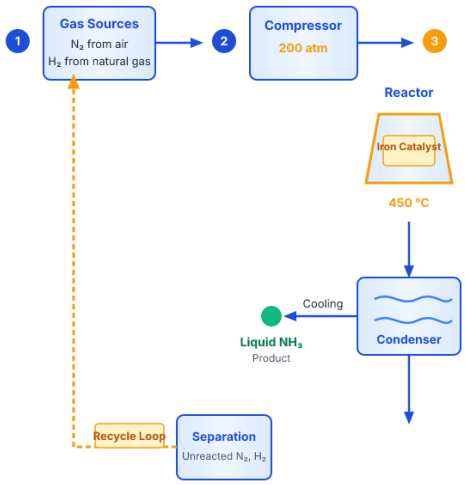


Figure 16: AI illustration of the Haber process plant: five numbered parts that the linked page reveals in order.

Appendix B: Project Timeline

Table 13: Project timeline.

Period	Phase	Main work
Sep 2025	Problem selection	Group formation, problem analysis, concept note, plan
Oct–Dec 2025	Research and design	Literature survey, competitor study, requirements, architecture, UI plan

Period	Phase	Main work
Jan–Mar 2026	Prototype V1	Parsing, AI rewriting, engagement score, quiz generation, Streamlit demo
Apr–Jul 2026	Learnova V2	FastAPI and React platform, decision engine, builders, accounts, library, tutor
Aug–Oct 2026	Real decks and research	Slide reader and OCR, illustrations, explainers, laboratories, AI key pool, papers, review

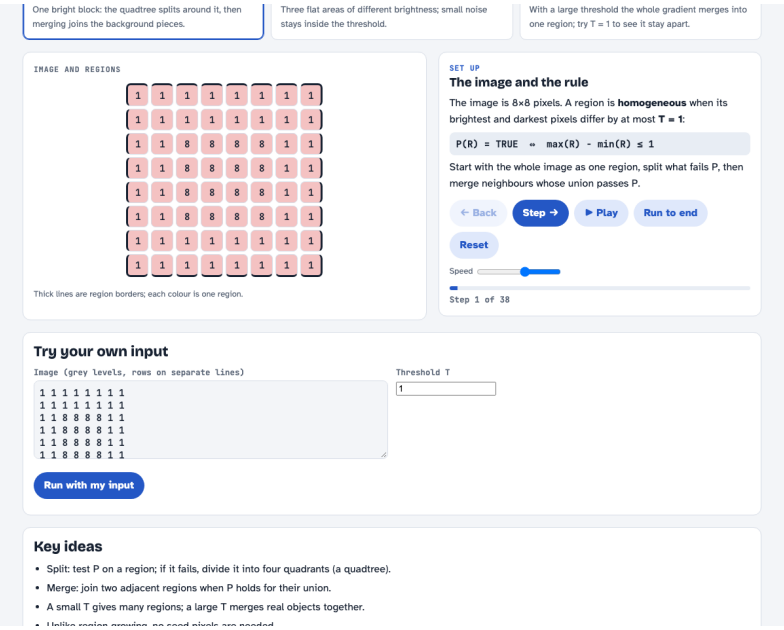


Figure 17: Region Splitting and Merging laboratory built on the shared laboratory bench.

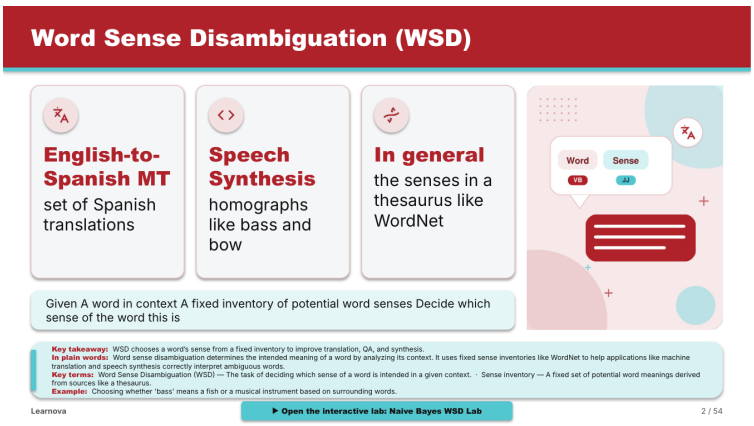


Figure 18: After: the same slide rebuilt with structured cards, an AI-drawn diagram, a quiz and a laboratory link.